Handoff: deploy "Cat Clash: Yarn Wars"

Hi Claude Code. This folder is a finished browser game. Please put it online as a public website and give me the link. Nothing needs a redesign, and the game logic doesn't need to change. The job is to **deploy it and make sure online multiplayer works on the live site.**

What's in this folder

```
cat-clash/
├── public/index.html   ← the whole game (one self-contained HTML file: canvas + JS, no build step)
├── server.js           ← small Node server: serves public/ AND relays online-multiplayer messages over WebSocket at /ws
├── package.json        ← one dependency: "ws". Start command: npm start
├── .gitignore
└── HANDOFF.md          ← this file
```

- **Node 18+** is required. The server listens on `process.env.PORT` and falls back to 3000.
- `GET /catclash-health` returns `{"catclash":true}`. The game calls this when it loads to find out that it's on its own server, then switches online play over to the WebSocket at `/ws`. Keep this endpoint and that path.
- Without the server, the game still works on one device: 1–4 players can share a keyboard. **Online play needs `server.js` to be running**, so a static-only host isn't enough on its own.

How online play works (so you know what to protect)

- One player is the **host**: their browser runs the game simulation. Guests send only their inputs.
- Players meet in a "room" named `catclash-<4-letter code>`. Every client keeps a small "presence" JSON object (≤ ~4 KB). The host's presence carries game snapshots about 30 times a second, and each guest's presence carries its controls.
- `server.js` is only a relay. It forwards each client's presence to the other clients in the same room: `join`, then `presence`, then `leave`. New joiners get a `snap` of everyone, plus `p` updates and `left` notices. It has no game logic.

- **The host needs a persistent WebSocket connection.** Serverless or function-only hosting (plain Vercel or Netlify functions) will *not* work for `/ws`. Use a host that runs a long-lived Node process.

Steps

1. **Run it locally first** `bash npm install npm start # → http://localhost:3000` Open the URL in two browser windows (or one normal and one incognito). In window A choose **Play Online → Host a Game** and pick a cat. In window B choose **Play Online → Join a Game**, type the 4-letter code and pick a cat. Back in A, press **Start**. Moving in B should move B's cat in both windows.

2. **Put it on GitHub.** Create a new repo (for example `cat-clash`), commit everything except `node_modules`, and push.

3. **Deploy it as a Node web service with WebSocket support.** Any of these works. Pick whichever I already have an account on, or ask me:

4. **Render** (free tier is fine): New → Web Service → connect the repo. Build command `npm install`, start command `npm start`. Free instances sleep when idle, so the first load can take about 30 s.
5. **Railway**: New Project → Deploy from GitHub repo. It detects Node and uses `npm start` automatically.
6. **Fly.io**: `fly launch` (accept the Node defaults), then `fly deploy`. Make sure the internal port matches `PORT`.

No environment variables or database are needed.

1. **Check the live site** (please actually do this, don't assume):
2. `https://<your-app>/catclash-health` returns `{"catclash":true,...}`.
3. The game loads, and **1 Player** is playable with the keyboard (WASD / arrows, J or Space claw, K yarn, L special).
4. **2 Players** on one keyboard works: P1 uses WASD + F/G/E, and P2 uses the arrow keys + / . and Right Shift.
5. **Online**: host in one browser and join from a second browser or a phone. The guest's cat moves on both screens, and the game goes through to the Yarn Shop after level 1.

6. The browser console shows no errors.

7. **Tell me the public URL.** I'll send it to friends, and they open it, press **Play Online → Join**, and type my code.

Good to know / possible follow-ups (only if asked)

- The page loads the "Fredoka" font from Google Fonts. If that's blocked, it falls back to system fonts, which is fine.
- Online rooms only exist while someone is in them. The server keeps no other state and doesn't need a database.
- Rooms are capped at 8 connections and messages at 8 KB in `server.js`. The game itself allows up to 4 cats per room.
- A custom domain can be added in the hosting provider's settings if I want one later.
- The same game also runs as a Claude artifact. There, online play uses Claude's built-in room feature instead of `/ws`. Please leave that code path in `index.html` (`wsRoomFactory` and the `window.claude` check) alone.